

Introduction au backend & Node.js

Séance 2 - On reprend le web, tranquillement, côté JavaScript

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

Comment marche le web - HTML/CSS/JS - Rappel JavaScript - Node.js - Rappel BD (PostgreSQL/Prisma/Neon)

Objectifs de la séance

On repart sur de bonnes bases, ensemble

À la fin de la séance, vous saurez :

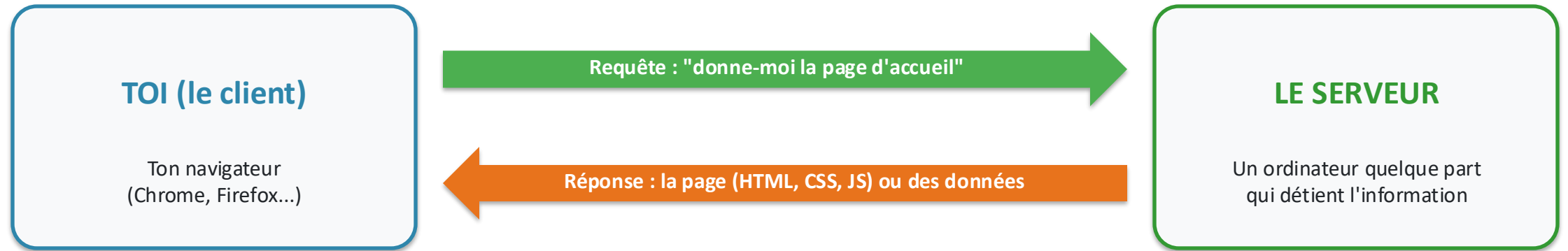
- **Expliquer comment marche le web** : client, serveur, navigateur
- **Situer les 3 langages du web** : HTML, CSS et JavaScript
- **Relire du JavaScript sans stress** : variables, fonctions, tableaux, objets
- **Utiliser quelques outils modernes du JS** : fonctions fléchées, map/filter, destructuring
- **Comprendre ce qu'est Node.js** et le situer dans la feuille de route du cours
- **Voir comment Node parle HTTP** (la base d'un backend)
- **Revoir les bases d'une BD relationnelle** : PostgreSQL, Neon, Prisma

PARTIE 1

Comment fonctionne le web ?

Le web : une conversation

Le client demande, le serveur répond



C'est tout le web en une phrase :

- **Tu tapes une adresse** -> ton navigateur envoie une demande (requête)
- **Le serveur reçoit la demande** -> il prépare une réponse
- **Le navigateur reçoit la réponse** -> il l'affiche joliment à l'écran

Les 3 langages d'une page web

Chacun a son rôle

HTML - la structure

- Le squelette de la page
- Titres, paragraphes, images, liens
- Comme les murs d'une maison
- `<h1>`, `<p>`, `<img>`, `<a>`

CSS - le style

- L'apparence de la page
- Couleurs, polices, mise en page
- Comme la peinture et la déco
- couleur, taille, position

JavaScript - le comportement

- Rend la page vivante
- Réagit aux clics, met à jour le contenu
- Comme l'électricité et la plomberie
- C'est NOTRE langage !

```
<!-- HTML : la structure -->
<h1>Bonjour la classe</h1>
<button>Cliquez-moi</button>

/* CSS : le style */
h1 { color: orange; font-size: 40px; }

// JavaScript : le comportement
button.onclick = () => alert("Bravo !");
```

WEB

Le rôle de JavaScript

Le langage qui rend tout dynamique

HTML et CSS affichent une page figée. JavaScript la rend interactive : il peut réagir aux actions de l'utilisateur et modifier la page en direct, sans la recharger.

Ce que JavaScript permet

- Réagir à un clic, à une saisie
- Afficher / cacher des éléments
- Valider un formulaire
- Aller chercher des données (API)
- Mettre à jour la page en direct

Exemples du quotidien

- Le bouton « j'aime » qui change
- Le panier qui se met à jour
- La recherche qui suggère
- La carte interactive
- Le chat en direct

Le même langage, deux endroits :

- **Dans le navigateur (frontend)** : on l'a toujours connu là
- **Sur le serveur (backend)** : grâce à Node.js -> c'est la suite du cours !

PARTIE 2

On (re)découvre JavaScript

C'est quoi, JavaScript ?

Un langage accessible, pas à craindre

JavaScript est le langage de programmation du web. Créé en 1995, il est aujourd'hui partout : sites, applications, serveurs, mobiles... C'est l'un des langages les plus utilisés au monde, et l'un des plus accessibles pour débiter.

Facile à essayer

- Aucune installation pour débiter
- Console du navigateur (F12)
- Résultat immédiat

Pardonne les erreurs

- Pas de compilation stricte
- On corrige et on relance
- On apprend en testant

Partout utile

- Frontend (React)
- Backend (Node.js)
- Une compétence qui sert

Notre première instruction : afficher quelque chose avec `console.log`

```
console.log("Bonjour la classe !");  
console.log(2 + 3);           // affiche 5  
console.log("Note :", 85);    // affiche Note : 85
```

JS

Les variables : let et const

Stocker une information dans une boîte

Une variable est une boîte qui porte un nom et contient une valeur.

```
let age = 25;           // peut changer
const nom = "Sonia";    // ne change jamais

age = 26;               // OK, autorisé
// nom = "Alex";        // ERREUR (const)

console.log(nom);        // Sonia
console.log(age);        // 26
```

JS

Quand utiliser quoi ?

- **const** : par défaut (valeur fixe)
- **let** : si la valeur va changer
- **var** : ancien, à éviter aujourd'hui
- Un nom clair = du code lisible
- Règle simple : const d'abord,
- let seulement si nécessaire

Astuce : choisissez des noms parlants (age, nomComplet, totalPanier) plutôt que x, y, z. Le futur vous remerciera.

Les types de données

Quels genres d'informations existe-t-il ?

Type	Exemple	C'est quoi
string	"Bonjour"	du texte (entre guillemets)
number	42 / 3.14	un nombre
boolean	true / false	vrai ou faux
array	[1, 2, 3]	une liste (tableau)
object	{ nom: 'Sonia' }	un ensemble clé/valeur
null	null	vide volontaire
undefined	undefined	pas encore défini

```
let texte = "Bonjour";
let nombre = 42;
let actif = true;
let liste = [1, 2, 3];
let perso = { nom: "Sonia" };
```

JS

```
// Connaître le type :
console.log(typeof texte);
// "string"
console.log(typeof nombre);
// "number"
```

JavaScript devine le type tout seul : pas besoin de le préciser (on verra plus tard que TypeScript, lui, nous demandera d'être précis).

Le texte et les template literals

Assembler des chaînes proprement

On a souvent besoin de combiner du texte et des variables. Le moderne backtick (``) rend ça beaucoup plus lisible.

Ancienne façon (avec +) :

```
const nom = "Sonia";  
const age = 25;  
  
console.log(  
  "Je suis " + nom + ", " + age + " ans"  
);
```

JS

Template literal (recommandé) :

```
const nom = "Sonia";  
const age = 25;  
  
// backticks `` et ${ }  
console.log(`Je suis ${nom}, ${age} ans`);  
// Je suis Sonia, 25 ans
```

JS

- On utilise les backticks `` (à côté du 1, pas les guillemets normaux)
- On insère une variable avec \${ ... }
- Plus lisible, moins d'erreurs qu'avec des + partout

Les conditions : if / else

Prendre une décision dans le code

```
const note = 75;

if (note >= 60) {
  console.log("Réussi !");
} else {
  console.log("À refaire");
}
// Affiche : Réussi !
```

JS

Les comparaisons

- === égal (valeur ET type)
- !== différent
- > >= < <=
- && ET || OU
- Préférez toujours === à ==

Attention au piège classique :

```
console.log(5 === 5);    // true
console.log(5 === "5");  // false (number vs string)
console.log(5 == "5");   // true  (== à éviter, il convertit)
// Règle d'or : utilisez toujours ===
```

JS

Les boucles : répéter une action

for, for...of, while

Boucle for (classique) :

```
for (let i = 0; i < 3; i++) {  
  console.log("Tour " + i);  
}  
// Tour 0  
// Tour 1  
// Tour 2
```

JS

Boucle for...of (sur une liste) :

```
const fruits = ["pomme", "kiwi"];  
  
for (const fruit of fruits) {  
  console.log(fruit);  
}  
// pomme  
// kiwi
```

JS

Boucle while (tant que la condition est vraie) :

```
let compteur = 3;  
while (compteur > 0) {  
  console.log("Décompte : " + compteur);  
  compteur = compteur - 1;  
}  
// Décompte : 3 , 2 , 1
```

JS

for...of est souvent le plus simple pour parcourir une liste.

Les fonctions

Un bloc de code réutilisable

Une fonction est une recette : on lui donne des ingrédients (paramètres), elle nous rend un résultat (return).

```
// Définition
function additionner(a, b) {
  return a + b;
}

// Appel (utilisation)
const resultat = additionner(2, 3);
console.log(resultat); // 5
```

JS

À retenir

- `function` : mot-clé
- `( a, b )` : les paramètres (entrées)
- `return` : la valeur renvoyée (sortie)
- On l'appelle par son nom + `( )`
- On peut la réutiliser autant
- de fois qu'on veut

Pourquoi c'est utile : on écrit la logique une seule fois, puis on la réutilise. Moins de code, moins d'erreurs, plus facile à corriger.

Les fonctions fléchées (arrow functions)

Une écriture plus courte et moderne

C'est une autre façon d'écrire une fonction, très courante aujourd'hui (surtout en React). Le résultat est identique.

Fonction classique :

```
function double(x) {  
  return x * 2;  
}  
  
console.log(double(5)); // 10
```

JS

Fonction fléchée (équivalente) :

```
const double = (x) => x * 2;  
  
console.log(double(5)); // 10  
  
// => se lit « donne »
```

JS

- **Plus courte** : pratique pour les petites fonctions
- **La flèche =>** remplace le mot fonction
- **Si une seule ligne** : le return est implicite (automatique)
- **Très utilisée avec map / filter** et dans React (qu'on verra bientôt)

Les tableaux (arrays)

Une liste ordonnée de valeurs

```
const notes = [80, 90, 70];

console.log(notes[0]);    // 80  (le 1er)
console.log(notes.length); // 3   (taille)

notes.push(100); // ajoute à la fin
console.log(notes); // [80, 90, 70, 100]

notes[1] = 95; // modifier
console.log(notes); // [80, 95, 70, 100]
```

JS

Points clés

- Crochets [] pour un tableau
- On compte à partir de 0 !
- notes[0] = le premier
- .length = le nombre d'éléments
- .push() = ajouter à la fin
- Idéal pour des collections :
- notes, utilisateurs, produits...

Piège fréquent : le premier élément est à l'index 0, pas 1. Donc le 3e élément est notes[2].

Parcourir un tableau : forEach, map, filter

Les outils modernes (très utiles !)

```
const notes = [80, 90, 55, 70];
```

JS

```
// forEach : faire qqch pour chaque élément  
notes.forEach(n => console.log(n));
```

```
// map : transformer chaque élément  
const bonus = notes.map(n => n + 5);  
// [85, 95, 60, 75]
```

```
// filter : garder selon une condition  
const reussites = notes.filter(n => n >= 60);  
// [80, 90, 70]
```

Que fait chacune ?

- forEach : exécute une action
- (ne renvoie rien)
-
- map : crée un NOUVEAU tableau
- transformé (même taille)
-
- filter : crée un NOUVEAU tableau
- filtré (plus petit ou égal)

Ces 3 méthodes utilisent une fonction fléchée. On les retrouvera partout, notamment pour afficher des listes en React.

Les objets

Regrouper des informations liées

Un objet regroupe plusieurs informations sous forme de paires clé : valeur. Parfait pour décrire une « chose ».

```
const etudiant = {  
  nom: "Sonia",  
  age: 25,  
  actif: true  
};  
  
console.log(etudiant.nom); // Sonia  
etudiant.age = 26;         // modifier  
etudiant.ville = "Montréal"; // ajouter
```

JS

À retenir

- Accolades { } pour un objet
- Des paires cle: valeur
- On accède avec un point :
- etudiant.nom
- On peut modifier et ajouter
- C'est exactement la forme
- du JSON (vu en séance 1) !

Tableau vs objet : un tableau est une liste (ordre, index) ; un objet décrit une entité avec des propriétés nommées.

Le destructuring

Extraire des valeurs d'un coup

Le destructuring permet de sortir rapidement des valeurs d'un objet ou d'un tableau dans des variables. Très courant en JavaScript moderne et en React.

Sur un objet :

```
const etudiant = { nom: "Sonia", age: 25 };  
  
// Au lieu de etudiant.nom...  
const { nom, age } = etudiant;  
  
console.log(nom); // Sonia  
console.log(age); // 25
```

JS

Sur un tableau :

```
const couleurs = ['rouge', 'vert'];  
  
const [premier, second] = couleurs;  
  
console.log(premier); // rouge  
console.log(second); // vert
```

JS

Pourquoi c'est pratique : du code plus court et plus clair. Vous verrez cette syntaxe constamment dans React (ex : `const [valeur, setValeur] = useState()`).

Ce qu'on retient du JavaScript

Vous avez déjà l'essentiel !

Stocker

- let / const
- string, number, boolean
- tableaux [] et objets { }

Décider & répéter

- if / else, comparaisons ===
- boucles for / for...of
- && (ET), || (OU)

Organiser & transformer

- fonctions + fonctions fléchées
- map / filter / forEach
- destructuring, template literals

Le secret : on apprend la programmation en pratiquant, pas en mémorisant.

Testez, cassez, corrigez. C'est normal de se tromper : c'est comme ça qu'on progresse.

PARTIE 3

Node.js : notre boîte à outils backend

Le problème de départ

JavaScript était prisonnier du navigateur

Pendant longtemps, JavaScript ne pouvait s'exécuter QUE dans un navigateur web. Pratique pour rendre les pages vivantes... mais très limité pour faire un vrai backend.

Dans le navigateur, JS peut

- Réagir aux clics
- Modifier la page affichée
- Afficher des messages (alert)
- Faire des animations
- Valider un formulaire

Mais il NE pouvait PAS

- Lire/écrire un fichier sur le disque
- Se connecter à une base de données
- Créer un serveur web
- Gérer des utilisateurs et des mots de passe
- Bref : pas de vrai backend

La question : et si on pouvait utiliser JavaScript AUSSI sur le serveur, pour bâtir le backend, sans apprendre un nouveau langage ?

La solution : Node.js

Le même JavaScript, mais côté serveur

Node.js (créé en 2009) permet d'exécuter du JavaScript en dehors du navigateur, directement sur un ordinateur ou un serveur. Le même langage que vous venez de revoir sert maintenant à construire le backend.

Même langage

- Variables, fonctions, tableaux...
- Exactement ce qu'on a vu
- Rien de nouveau à réapprendre

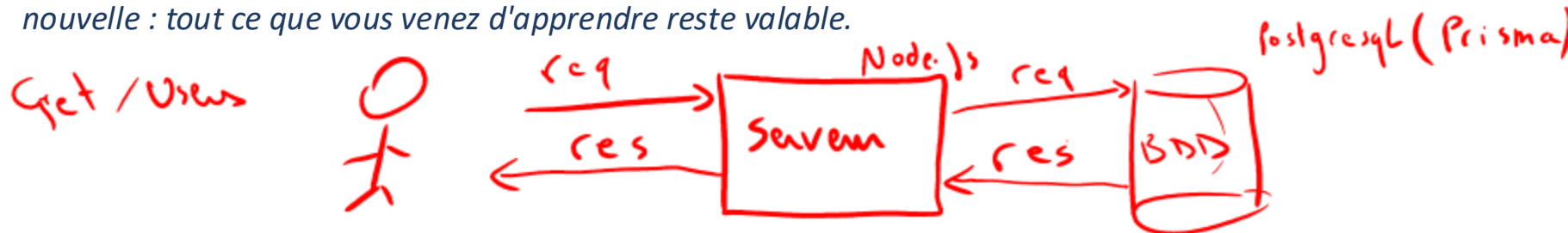
Nouveaux super-pouvoirs

- Lire / écrire des fichiers
- Se connecter à une BD
- Créer un serveur web

Un seul langage

- Frontend en JS ✓ *React*
- Backend en JS ✓ *Node.js*
- On dit : full-stack JavaScript
↳ Next.js

En résumé Node.js ce n'est pas un nouveau langage. C'est juste un endroit supplémentaire où votre JavaScript peut tourner. La bonne nouvelle : tout ce que vous venez d'apprendre reste valable.



Feuille de route backend du cours

Les outils qu'on va découvrir progressivement

On ne va PAS tout voir aujourd'hui. Voici la liste, pour savoir où on va :

Outil	Ce que ça fait	Quand on l'aborde
Node.js ✓	Exécute le JavaScript côté serveur	Maintenant
npm ✓	Installe des bibliothèques (packages)	Très bientôt
http (natif) ✓	Parler HTTP, créer un mini-serveur	Pour comprendre les bases
axios ✓	Appeler des API externes depuis Node	En cours de route
TypeScript ✓	Du JavaScript avec des types (plus sûr)	Progressivement, vers le milieu
Express ✓	Cadre simple pour bâtir une API	Une fois Node maîtrisé
PostgreSQL ✓	Notre base de données (relationnelle)	Quand on persiste des données
Neon	PostgreSQL hébergé dans le cloud	Avec PostgreSQL
Prisma	Pont entre notre code et la BD (ORM)	Avec la BD
Clerk ✓	Authentification (login, inscription)	Plus tard dans la session

Chaque outil arrivera au moment où il devient utile, pas avant.

Installer Node.js

Quelques minutes, une seule fois

Étape 1 - Télécharger la version LTS (stable) :

`https://nodejs.org` -> bouton LTS -> installer (Suivant, Suivant...)

WEB

Étape 2 - Vérifier dans le terminal (PowerShell, Terminal VS Code...) :

```
node -v  
npm -v
```

bash

```
v22.18.0  
10.9.3
```

résultat

Bon à savoir

- LTS = Long Term Support = la version stable recommandée
- npm s'installe automatiquement avec Node (on l'explorera plus tard)
- Si les commandes affichent un numéro de version -> tout fonctionne !
- On installera aussi VS Code (éditeur de code) pour écrire confortablement

Votre premier programme Node.js

Exécuter un fichier JavaScript

1. Créez un fichier nommé `app.js` avec ce contenu :

```
console.log("Mon premier programme Node !");  
  
const nom = "la classe";  
console.log(`Bonjour ${nom}`);
```

app.js

2. Lancez-le depuis le terminal (dans le bon dossier) :

```
node app.js  
  
Mon premier programme Node !  
Bonjour la classe
```

bash

Astuce : tapez simplement « node » (sans fichier) pour ouvrir le REPL, une console interactive parfaite pour tester rapidement une idée.

Comment Node parle HTTP

Le module `http` natif (sans framework)

HTTP est le protocole du web (vu en séance 1 : requête + réponse). Node intègre d'origine un module `http` qui sait écouter les requêtes et envoyer des réponses. C'est la fondation sur laquelle tout backend Node est bâti.

```
// fichier : serveur.js
const http = require('http');

const serveur = http.createServer((req, res) => {
  res.writeHead(200, {
    'Content-Type': 'application/json'
  });
  res.end(JSON.stringify({ msg: 'Bonjour !' }));
});

serveur.listen(3000); // écoute sur le port 3000
```

JS

Les notions clés

- `require('http')` : module natif ✓
- `createServer(...)` : créer un serveur
- `req` : la requête entrante
- `res` : la réponse à renvoyer
- `writeHead` : statut + en-têtes
- `res.end` : terminer + envoyer
- `listen(3000)` : ouvrir un port

Lourd à écrire à la main pour de vraies API ! C'est pour cela qu'on utilisera plus tard des outils plus simples (Express, puis Next.js). Mais c'est bien Node qui fait le vrai travail en dessous.

Navigateur vs Node.js

Le même langage, deux environnements

	Navigateur	Node.js
Exécute du JavaScript	Oui	Oui ✓
document, window (page)	Oui	Non
alert(), boutons, DOM	Oui	Non
Lire / écrire des fichiers	Non	Oui ✓
Créer un serveur web	Non	Oui ✓
Se connecter à une base de données	Non	Oui ✓

À retenir : le langage est le même, mais l'environnement change. Dans le navigateur, JS manipule la page. Dans Node, JS manipule le système (fichiers, réseau, BD). C'est ce qui en fait un outil de backend.

PARTIE 4

Rappel : les bases de données

Pourquoi une base de données ?

Au-delà des fichiers texte

Une base de données (BD) est un endroit organisé pour stocker, retrouver et protéger des informations. C'est ce qui permet à une application de se souvenir des choses entre deux visites.

Sans base de données

- Tout disparaît en redémarrant
- Difficile à chercher
- Pas de partage entre utilisateurs
- Pas de cohérence garantie

Avec une base de données

- Persistance (durée)
- Requêtes rapides et puissantes
- Plusieurs utilisateurs en sécurité
- Sauvegardes, intégrité, fiabilité

Analogie : un classeur intelligent. Tu y déposes des fiches (commandes, utilisateurs, articles), tu peux les retrouver en un clin d'oeil grâce à un langage de requête, et elles restent là même quand tu fermes la lumière.

Les bases relationnelles

Tout est rangé dans des tableaux

Une BD relationnelle organise les données en tables (comme des onglets Excel). Chaque table décrit un type d'entité.

Exemple : table Users

id (PK)	nom	email
1	Sonia	sonia@cmaisonneuve.ca
2	Alex	alex@cmaisonneuve.ca
3	Maxime	max@cmaisonneuve.ca

Handwritten red annotations:
- A red arrow points from the word "Attribut" to the "email" column header.
- A red circle highlights the "email" column header.
- A red oval highlights the row for Alex (id 2).

Le vocabulaire

- Table : un grand tableau
- Ligne (row) : un enregistrement
- Colonne : une propriété (avec un type)
- Clé primaire (PK) : identifiant unique
- Clé étrangère (FK) : lien vers
- une autre table

Plusieurs tables peuvent être liées : par exemple une table Posts où chaque ligne a un `userId` qui pointe vers la table Users. C'est ce qu'on appelle « relationnel ».

SQL en 4 commandes essentielles

Parler à la BD = le CRUD

SQL (Structured Query Language) est le langage pour interroger une BD relationnelle. Avec 4 commandes, on couvre déjà 90 % des besoins :

```
-- Lire
SELECT * FROM Users;
SELECT nom FROM Users WHERE id = 1;

-- CREER
INSERT INTO Users (nom, email)
VALUES ('Lina', 'lina@col.ca');

-- MODIFIER
UPDATE Users
SET email = 'nouv@col.ca'
WHERE id = 2;

-- SUPPRIMER
DELETE FROM Users WHERE id = 3;
```

Read

Create

Update

Delete

CRUD

SQL

Action	SQL	REST (rappel)
Lire	SELECT	GET
Créer	INSERT	POST
Modifier	UPDATE	PUT / PATCH
Supprimer	DELETE	DELETE

C'est exactement le CRUD vu en séance 1 ! Une API REST est souvent une traduction directe de ces commandes SQL.

PostgreSQL & Neon

Notre BD relationnelle + son hébergement cloud

PostgreSQL (la BD)

- BD relationnelle libre et gratuite
- Robuste, mature, très utilisée en industrie
- Supporte le SQL standard + extensions modernes
- Excellente pour les projets web sérieux
- Documentation officielle : postgresql.org

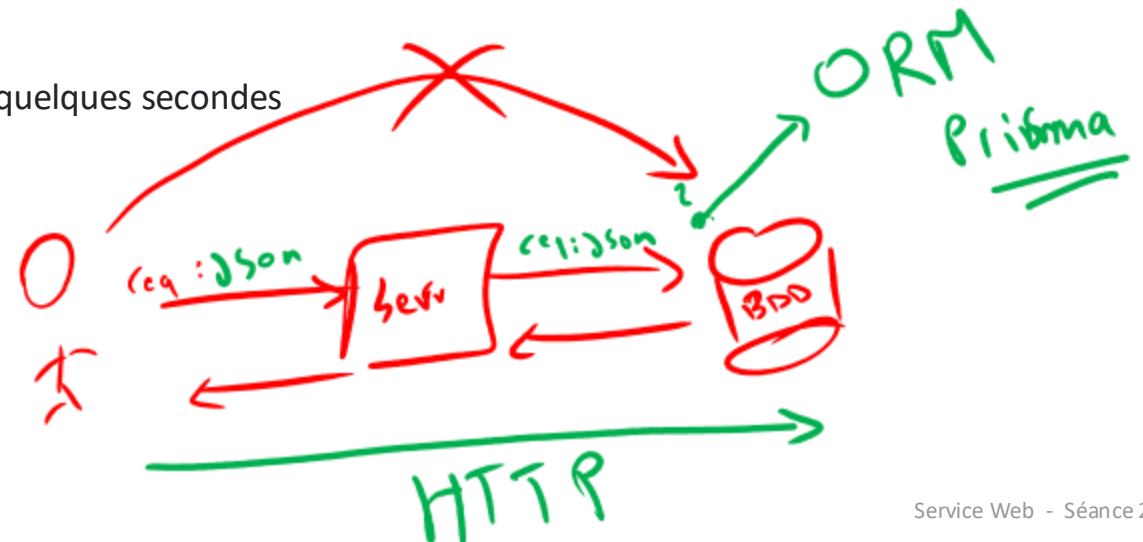
Transactions!

Neon (l'hébergement)

- PostgreSQL prêt à l'emploi dans le cloud ✓
- Aucune installation locale nécessaire
- Plan gratuit largement suffisant pour le cours
- On crée un projet -> on reçoit une URL de connexion ✓ *env*
- Site : neon.tech

Concrètement :

- On crée un projet sur Neon -> Neon nous donne une BD PostgreSQL en quelques secondes
- On copie l'URL de connexion (DATABASE_URL) dans notre projet Node
- Notre code Node parle à cette BD comme si elle était à côté



Prisma - le pont entre Node et la BD

Un ORM qui simplifie tout

Prisma est un ORM (Object-Relational Mapper) : il permet de parler à la BD directement en JavaScript / TypeScript, sans écrire de SQL à la main. Plus lisible, plus sûr, plus rapide à développer.

Sans Prisma (SQL brut) :

```
SELECT * FROM Users  
WHERE email = 'sonia@col.ca';
```

SQL

Avec Prisma (du JavaScript) :

```
await prisma.user.findUnique({  
  where: { email: "sonia@col.ca" }  
});
```

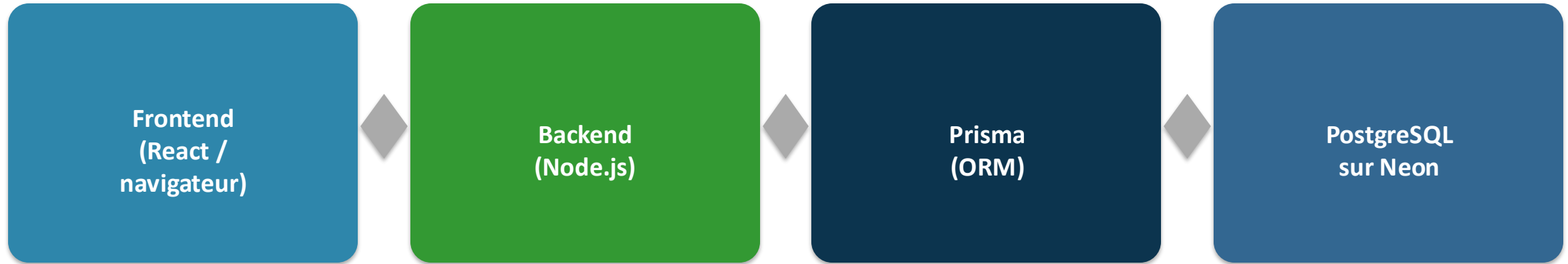
JS

Ce que Prisma nous apporte

- Un fichier `schema.prisma` qui décrit la BD (modèles)
- Migrations automatiques : `prisma migrate dev`
- Types TypeScript générés automatiquement
- Auto-complétion intelligente dans VS Code
- Fonctionne très bien avec PostgreSQL + Neon

Le schéma complet

Comment tout s'assemble



À chaque niveau, son rôle :

- **Frontend** : ce que voit l'utilisateur (React)
- **Backend Node.js** : reçoit les requêtes, applique la logique métier
- **Prisma** : traduit notre code JS en SQL pour la BD
- **PostgreSQL sur Neon** : stocke et restitue les données en toute sécurité

Récapitulatif de la séance

Ce qu'il faut retenir

Le web

- Client <-> serveur
- Requête, réponse
- HTML / CSS / JS

JavaScript

- let / const, types
- fonctions (+ fléchées)
- tableaux, objets
- map / filter / destructuring

Node.js

- JS hors navigateur
- node app.js
- Module http natif
- Base de notre backend

Base de données

- Tables, lignes, clés
- SQL : SELECT, INSERT...
- PostgreSQL + Neon
- Prisma : le pont

Vous avez la carte du territoire.

Maintenant on va explorer chaque outil à son rythme : Node, npm, axios, TypeScript, Express, Prisma, PostgreSQL, Neon...

Pour la prochaine séance

On continue en douceur

À faire de votre côté

- Installer Node.js (LTS) + VS Code
- Vérifier : `node -v` et `npm -v`
- Créer et lancer un `app.js`
- Rejouer 2-3 exemples JS de la séance
- Créer un compte Neon (gratuit)
- Curiosité : visiter prisma.io

Au programme ensuite

- Manipuler des données en JavaScript
- L'asynchrone en douceur (`await`)
- `npm` : installer notre première librairie
- `axios` : appeler une API depuis Node
- Premier mini-projet Node
- Plus tard : Express, Prisma, BD...

Notre fil conducteur (90 h) : Backend Node.js (avec Express, axios, Prisma, PostgreSQL/Neon) -> Frontend React -> Full-stack Next.js, avec TypeScript introduit progressivement. Aucune précipitation : on construit brique par brique.



Questions ?

Aucune question n'est bête. On avance ensemble, à votre rythme.

Testez, cassez, corrigez - c'est comme ça qu'on apprend.